

Protocol Execution Flow: Voter → Aggregation

Phase 1 — Setup (Trusted Third Party TP)

TP chooses safe primes $p, q \rightarrow N = p \cdot q$
TP picks $H \in \mathbb{Z}^*_{N^2} \rightarrow$ publishes (N, H) , goes offline
Aggregator A picks $sk_A \in \mathbb{Z}^*_{N^2} \rightarrow$ publishes $pk_A = H^{\{sk_A\}} \bmod N^2$
Each voter U_i picks $sk_i \in [0, N^2)$ independently (no key dealer)

Phase 2 — Voter U_i (Browser/WASM)

1. Select candidate $\rightarrow$ vote vector v_i
2. Pack: $m_i = \sum v_{i,j} \cdot 2^{\{25(k-j)\}}$
3. Encrypt: $c_i = H^{\{sk_i\}} \cdot (1+N)^{\{m_i\}} \bmod N^2$
4. Compute: $aux_i = pk_A^{\{sk_i\}} \bmod N^2$

(sk_i serves as both secret key and randomness r)
Performed in WebAssembly (libtommath, Emscripten)

store c_i, aux_i

Phase 3 — Collector C

Receives all aux_i from voters
Computes: $aux = \prod aux_i \bmod N^2$
 $= H^{\{sk_A \cdot \sum sk_i\}} \bmod N^2$
Sends $aux \rightarrow$ Aggregator A

Phase 4 — Aggregator A (Native libtommath C Module)

Input: ciphertexts $\{c_1, \dots, c_n\}$, aux , (N, H, sk_A)

Step 1. Product: $\Pi = \prod c_i \bmod N^2 = (1 + S \cdot N) \cdot H^{\{\sum sk_i\}} \bmod N^2$
where $S = \sum x_i$ (binomial expansion in $\mathbb{Z}_{N^2}$)

Step 2. Exponentiate: $\Pi^{\{sk_A\}} \bmod N^2 = (1 + S \cdot N)^{\{sk_A\}} \cdot H^{\{sk_A \cdot \sum sk_i\}} \bmod N^2$

Step 3. Cancel mask: $P = \Pi^{\{sk_A\}} \cdot aux^{\{-1\}} \bmod N^2 = (1 + S \cdot N)^{\{sk_A\}} \bmod N^2$
 $aux^{\{-1\}}$ exists because $\gcd(H, N^2) = 1$ (N is valid semiprime)

Step 4. L-function: $L(P') = (P' - 1) / N = sk_A \cdot S \bmod N$
 $(1 + S \cdot N)^{\{sk_A\}} \equiv 1 + sk_A \cdot S \cdot N \bmod N^2$ (higher terms vanish)

Step 5. Recover sum: $R = L(P') \cdot sk_A^{\{-1\}} \bmod N = S \bmod N$
 $sk_A^{\{-1\}}$ exists because $\gcd(sk_A, N) = 1$ by construction

Step 6. Unpack: For $j = 1..k$: $count_j = (R \div 25(k-j)) \bmod 2^{25}$
25-bit slots, reverse order (candidate 1 at MSB)

Phase 5 — Result

$R = \sum x_i = \sum_{i=1}^n \sum_{j=1}^k v_{i,j} \cdot 2^{\{25(k-j)\}}$
Vote counts: $[count_1, count_2, \dots, count_k]$ where $count_j = \sum_i v_{i,j}$

Assumptions for correctness:

- (A1) $N = p \cdot q$ where p, q are odd primes $\rightarrow \gcd(aux, N^2) = 1, \gcd(sk_A, N) = 1$
 - (A2) $sk_A \neq H$ (non-degenerate key) $\rightarrow pk_A \neq H$, decryption non-trivial
 - (A3) $S = \sum x_i < N \rightarrow$ modular reduction is exact, $R = S$ (no wraparound)
 - (A4) $k \cdot 25 < \log_2 N \rightarrow$ packed votes fit in single Paillier plaintext
 - (A5) Each $v_{i,j} \in \{0,1\} \rightarrow$ slot sums $\sum v_{i,j} < 2^{25}$ (no overflow per candidate)
 - (A6) $sk_i = r_i$ (voter uses secret key as encryption randomness) $\rightarrow aux_i = pk_A^{r_i}$ cancels H^{r_i}
- Under DCR assumption: individual c_i and aux_i are semantically secure (indistinguishable from random in $\mathbb{Z}^*_{N^2}$)